

Gateway — N100 网关 + 多层过滤 (双网口 / 单网口)

方案选择:

- **双网口设备 (N100 软路由/多网口主机)?** 按下方主流程 (串联网关, 推荐)
- **单网口设备 (旧电脑/NAS/树莓派)?** 跳转到 [单网口旁路网关方案](#)

项目背景

家庭路由器 (Cudy WR3600E) 自带黑白名单功能不稳定, 对纯 IP 地址 访问经常拦截失效。孩子使用的安卓平板需要可靠的网络访问管控。

需求清单

编号	需求	实现方式
R1	域名级黑/白名单, Web GUI 管理	AdGuard Home (Port 80)
R2	拦截纯 IP 直连访问	nftables IP 黑名单 + SNI 检查
R3	防止通过修改 DNS 绕过	nftables 强制劫持 UDP/TCP 53 → 本机
R4	防止 DoH/DoT 绕过	nftables 封锁 853 + AdGuard DNS 劫持
R5	查看孩子设备的历史访问记录	AdGuard Home 查询日志
R6	按设备查看上网流量/活跃时长	ntopng (Port 3001)
R7	按时间段限制孩子可上网时段	AdGuard Home 客户端时间表
R8	成人内容 + 社交媒体拦截	AdGuard 过滤器 + nftables SNI
R9	其他家庭成员不受影响 (分组策略)	AdGuard kids/default 客户端分组
R10	低成本硬件, 国内易购, 长期可靠	N100 小主机 + Debian 12

设计原则

1. **多层纵深防御** — DNS 被绕过还有 IP 黑名单, IP 被绕过还有 SNI 检查, 单点失效不会导致全局暴露
2. **Web GUI 优先** — 使用者是家长, 非技术人员, 日常操作 (查看记录、修改白名单、设时间表) 全部通过浏览器完成
3. **Docker 解耦** — 每个功能独立容器, 替换/升级无影响, 和宿主机 nftables 组合形成完整方案
4. **性价比优先** — N100 小主机 ~¥600, 功耗 ~10W, 性能可同时跑 网关 + 监控 + 未来扩展 (Home Assistant、NAS 等)

硬件兼容性

支持

任何能装标准 Linux 的 x86_64 / ARM64 设备均可:

类别	举例	适用模式
x86 软路由/小主机	N100, N95, J4125	双网口(串联) 或 单网口(旁路)
ARM 开发板	NanoPi R4S/R2S, 树莓派 4/5, Orange Pi	双网口 或 单网口(旁路)
旧 PC/笔记本	退役台式机/笔记本 + USB 网卡	视网口数而定
虚拟机	Proxmox/ESXi/VMware 上的 Linux VM	双网口(直通/桥接) 或 单网口(旁路)

不兼容: macOS / Mac Mini

macOS (包括搭载 M 芯片或 Intel 的 Mac Mini) 不能用作此方案的网关设备。

问题	详情
Docker network_mode: host	macOS 上 Docker 运行在 Hypervisor 层的 Linux VM 中, 容器和宿主机不在同一网络命名空间。network_mode: host 不生效, AdGuard 无法直接在宿主机网口上监听 53 端口
DNS 劫持无法实现	nftables 的 redirect to :53 规则将 DNS 流量转向宿主机网口的 53 端口。macOS 不具备 Linux 的 nftables, macOS 自身的 pf 防火墙不支持 redirect 表达式和 SNI 匹配
SNI 过滤不可用	macOS 的 pf 是 BSD 防火墙, 没有 tls sni 匹配能力, 无法在 TLS 握手阶段按域名拦截
核心依赖全在 Linux 内核	nftables + redirect + SNI match + netdev ingress + ipset 全部是 Linux 内核特性, macOS 一个都不支持

变通方案:

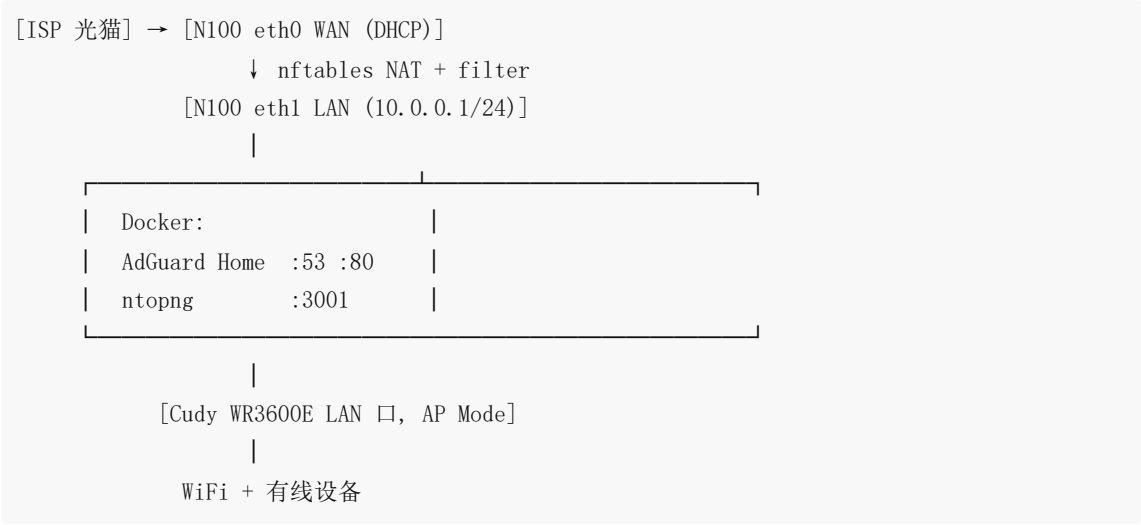
方法	评估
Mac Mini 格掉装 Ubuntu/Debian (仅 Intel 机型)	✅ 可行, 方案完全一致
Mac Mini 上跑 Linux 虚拟机 (UTM/Parallels)	⚠️ 仅能走单网口旁路, 需多一层 bridge 配置, 不推荐
Mac Mini 只做监控面板 (AdGuard + ntopng)	⚠️ Web GUI 可用, 但过滤和 DNS 劫持全丢失, 只剩半套方案
放弃 Mac Mini, 另购 N100 小主机	✅ ~¥600, 功耗 10W, 完整功能

如果你手头只有 Mac Mini: Intel 版本建议直接格掉装 Ubuntu/Debian, M 芯片版本建议拿来干别的事, 网关另用 N100 或 NanoPi R4S。

硬件

设备	角色	网口
N100 小主机	网关 + Docker Host	1 WAN, 1 LAN
Cudy WR3600E	纯 WiFi AP	接 N100 LAN 口

拓扑



目录结构

```
/opt/gateway/
├── docker-compose.yml
├── nftables.conf           # 双网口防火墙规则
├── nftables-single-nic.conf # 单网口防火墙规则
├── scripts/
│   ├── 01-setup-host.sh
│   ├── 02-nftables-apply.sh
│   ├── 03-verify.sh
│   └── single-nic/
│       ├── 01-setup-host.sh
│       ├── 02-nftables-apply.sh
│       └── 03-verify.sh
├── adguard/
│   ├── work/      (bind mount, 自动创建)
│   └── conf/      (bind mount, 自动创建)
└── ntopng/
    └── data/      (bind mount, 自动创建)
```

快速部署 (7 步)

Step 1: N100 装好 Debian 12

```
# 装好 Debian 后，更新并安装依赖
apt update && apt upgrade -y
apt install -y docker.io docker-compose-v2 nftables curl

# 确保当前用户能跑 docker
usermod -aG docker $USER
newgrp docker
```

Step 2: 把本项目拷到 N100

```
scp -r gateway/ root@n100.lan:/opt/
```

或者在 N100 上 `git clone` / U 盘拷贝。

Step 3: 配置网口

```
cd /opt/gateway/scripts
chmod +x *.sh
bash 01-setup-host.sh
```

脚本会：

- 自动检测所有网口并让你确认哪个是 WAN / LAN
- 配置 WAN (DHCP) + LAN (静态 10.0.0.1/24)
- 启用 IP 转发
- 自动更新 `nftables.conf` 和 `docker-compose.yml` 中的接口名

Step 4: 加载防火墙

```
bash 02-nftables-apply.sh
```

Step 5: 启动 Docker 容器

```
cd /opt/gateway
docker compose up -d

# 确认
docker compose ps
```

Step 6: 配置 AdGuard Home

1. 浏览器打开 `http://10.0.0.1:3000` (首次设置向导)

2. 根据向导配置:

- 管理界面端口: **80**
- DNS 监听: **All interfaces**
- 上游 DNS: `223.5.5.5` `119.29.29.29`
- 设置管理员账号密码

3. 添加过滤器列表:

```
AdGuard DNS filter
HaGeZi - Multi PRO
OISD - Big
blocklistproject - Adult
blocklistproject - Gambling
blocklistproject - TikTok
blocklistproject - Facebook
```

4. 添加孩子设备:

- 设置 → 客户端设置 → 添加客户端
- MAC 地址: 平板 WLAN MAC
- 标签: `kids`
- 勾选 "屏蔽成人内容"
- 勾选 "使用安全搜索"
- 添加时间表 (例: 每天 21:00-07:00 拦截所有流量)

5. 配置 DHCP (可选):

- 如果 Cudy 关了 DHCP, 在这里开启 AdGuard 的 DHCP
- 范围: `10.0.0.100 - 10.0.0.200`
- 网关: `10.0.0.1`

Step 7: 配置 Cudy WR3600E (AP 模式)

1. 登录 Cudy 后台

2. 网络 → LAN:

- IP: `10.0.0.2`
- 掩码: `255.255.255.0`
- 网关: `10.0.0.1`
- DNS: `10.0.0.1`

3. DHCP 服务器: 关闭

4. 保存重启

5. 物理换线: N100 LAN 口 → Cudy 的** LAN 口** (非 WAN 口)

验证

```
bash scripts/03-verify.sh
```

或手动:

```
# Layer 1: DNS 域名拦截
nslookup pornhub.com 10.0.0.1    # 应返回 0.0.0.0

# Layer 2: DNS 强制劫持
nslookup baidu.com 8.8.8.8       # 仍能解析（被劫持到本机）

# Layer 3: IP 黑名单
ping <在黑名单里的IP>           # 应超时

# Layer 4: SNI 过滤
curl -k --resolve "tiktok.com:443:157.240.0.1" https://tiktok.com/
# 应 connection timeout/reset
```

日常运维

```
# 查看拦截统计
nft list ruleset | grep counter

# 查看防火墙日志（最近 100 条拦截）
journalctl -k --since "1 hour ago" | grep -E 'SNI|IP|DOT' | tail -100

# 重载防火墙
systemctl reload nftables-gateway

# 更新 AdGuard Home
docker compose pull && docker compose up -d

# 更新 ntopng
docker compose pull ntopng && docker compose up -d ntopng
```

Web 面板地址

面板	地址	用途
AdGuard Home	http://10.0.0.1	DNS 过滤, 查询日志, 黑白名单, 客户端规则
ntopng	http://10.0.0.1:3001	实时流量, per-device 统计, 活跃时长估算

设备管理

添加新设备到监控

- 1. 获取设备 MAC 地址:
 - 安卓平板: 设置 → 关于 → WLAN MAC 地址
 - iOS: 设置 → 通用 → 关于本机 → Wi-Fi 地址
- 2. AdGuard Home → 客户端 → 添加客户端:
 - 标识符: AA:BB:CC:DD:EE:FF
 - 标签: kids
 - 勾选 "屏蔽成人内容" + "使用安全搜索"
 - 设置时间表 (如: 每天 21:00-07:00 拦截)
- 3. 如需固定 IP (方便 ntopng 对应): DNS → DHCP → 静态租约 → 添加 MAC→IP 绑定
- 4. 自此该设备所有 DNS 查询走 kids 策略, **改 IP 不影响规则** (识别基于 MAC)

移除设备

- 1. AdGuard Home → 客户端 → 选中设备 → 删除
- 2. 设备退回 default 全局规则 (仅拦截广告/恶意软件, 不拦截社交/成人)
- 3. 若配过静态 DHCP 租约, 一并删除

查看某个设备的上网记录

- 1. AdGuard Home → 查询日志 → 客户端筛选 → 选目标设备
- 2. 显示该设备所有 DNS 查询时间线 (域名 / 时间 / 是否被拦截)
- 3. ntopng → Hosts → 选目标 IP → 查看实时流量 + 历史流量趋势

防绕过策略

本方案的核心防御点是**强制所有 DNS 查询经过 AdGuard Home**, 结合多层过滤应对各种绕过企图。

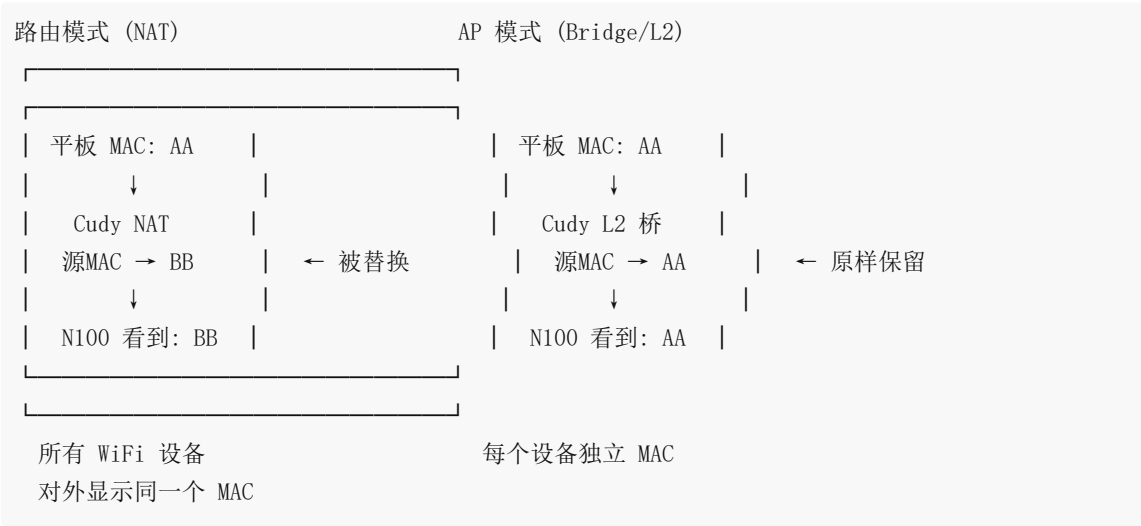
场景	是否可绕过	原因
改平板的 IP 地址	否	AdGuard 用 MAC 而非 IP 识别客户端; MAC 不变, 策略不变
改平板的 MAC 地址	否	新 MAC 不在 DHCP 注册表, 拿不到 IP (或落入 default 组, 无特殊权限)
手动设 DNS 为 8.8.8.8	否	nftables 劫持所有 53 端口流量 → AdGuard Home
使用 DoH (DNS over HTTPS)	部分	已知 DoH IP 被封锁 + AdGuard 劫持 53 端口生效
用 IP 直连不良网站	否	nftables SNI 检查: TLS 握手阶段读取 Server Name → 匹配黑名单 → DROP
使用 VPN/代理	部分	需手动添加 VPN IP 到 BLOCKED_IPS; 可封锁常见 VPN 端口 (1194/1723/500/4500)
改路由器配置还原	否	Cudy 已降级为纯 AP, DHCP/NAT/路由功能全部由 N100 接管

核心: MAC 识别, 非 IP 识别

AdGuard Home 通过 **MAC 地址**追踪设备。只要 AdGuard 的 DHCP 服务运行, 它维护的是 MAC→IP 映射表。**孩子改了 IP 地址不会逃出 kids 分组的规则。**

为什么 AP 模式下 MAC 透传有效

关键在于 Cudy 的工作模式:



路由器在 **路由/NAT 模式**下是三层设备, 会对出站帧做 SNAT/Masquerade, **源 MAC 被替换为路由器自身的 WAN 口 MAC**。此时 N100 看到的永远是 Cudy 的 MAC, 无法区分不同设备。

而 **AP 模式**下 Cudy 退化为纯二层交换机: WiFi 帧转以太网帧时 **源 MAC 地址原样保留**, 不做任何改写。N100 的 eth1 可以直接看到每个 WiFi 客户端的真实 MAC, MAC filter 完全有效。

部署后可在 N100 上验证:

```
ip neigh show dev eth1    # 应看到所有 WiFi 设备的 MAC + IP
```

可选: MAC 白名单 (严格模式)

如需拒绝未登记的陌生设备 (如孩子的第二台平板), 编辑 `nftables.conf`, 取消底部 `table netdev filter` 的注释。效果:

```
已登记 MAC → 正常上网 (走对应分组策略)
未登记 MAC → 所有流量 DROP, 连不上网
```

访客需家长手动在 nftables 中添加 MAC 才能联网。

常见 VPN 端口速查

如需封锁 VPN, 在 `nftables.conf` 的 `chain forward` 中追加:

```
# VPN 端口封锁
tcp dport { 1194, 1723, 500, 4500, 1701 } log prefix "VPN-blocked: " drop
```



```
udp dport { 1194, 500, 4500, 1701 } log prefix "VPN-blocked: " drop
```

然后 `systemctl reload nftables-gateway` 。

自定义

增加 SNI 黑名单域名

编辑 `nftables.conf`，在 `BLOCKED_SNI` 中添加域名，然后：

```
systemctl reload nftables-gateway
```

增加 IP 黑名单

编辑 `nftables.conf`，在 `BLOCKED_IPS` 中添加 IP/网段：

```
define BLOCKED_IPS = {  
    1.2.3.4,  
    5.6.7.0/24,  
}
```

按时间段控制 (AdGuard Home Web UI)

客户端设置 → 选择 kids 设备 → 时间表：

- 上学日: 07:00-08:00 / 18:00-21:00 (仅允许这些时段)
- 周末: 07:00-21:00

扩展监控 (可选)

```
# 安装 Grafana + Prometheus 做统一仪表盘  
docker compose -f docker-compose.extend.yml up -d
```

单网口旁路网关方案

当硬件只有一个网口 (老旧笔记本、NAS、树莓派、单网口小主机)，无法做串联网关时，使用此方案。

拓扑对比





关键差异

	双网口 (串联)	单网口 (旁路)
N100 角色	主路由 (NAT+DHCP+DNS)	旁路网关 (纯 filter, 路由器承担 NAT/DHCP)
NAT	N100 做 SNAT/Masquerade	路由器做 NAT, N100 不做
回程流量	必经 N100 (对称)	绕过 N100 直接回设备
Cudy 角色	纯 AP (DHCP 关闭)	正常路由 (DHCP 开启)
部署复杂度	需物理改线	只改路由器 DHCP 设置
DNS 劫持	✅	✅
SNI 拦截	✅	✅ (SYN 经过 N100)
IP 黑名单	✅	✅ (同上)
对外网影响	光猫断则全网断	N100 宕机不影响外网 (回退到路由器)

非对称路由的影响

出站: 平板 → N100 (过滤) → 路由器 → Internet ✅

回程: Internet → 路由器 → 平板 ⚠️

回程流量直接回到平板不经过 N100，但不影响过滤功能:

- **DNS** — AdGuard 收到查询后直接回复，请求/响应都在同一 L2
- **SNI** — TLS 握手首个 SYN 即被 DROP，连接无法建立
- **IP 黑名单** — 出站包命中即 DROP，无回程也无妨

部署步骤 (5 步)

Step 1: 装好系统 + 依赖

```
apt update && apt upgrade -y
apt install -y docker.io docker-compose-v2 nftables curl
```

```
usermod -aG docker $USER
```

Step 2: 拷入项目文件

```
# 把 gateway/ 目录放到 N100 上
scp -r gateway/ root@<n100-ip>:/opt/
cd /opt/gateway
```

Step 3: 配置网口 (单网口版)

```
cd /opt/gateway/scripts/single-nic
chmod +x *.sh
bash 01-setup-host.sh
```

脚本会:

- 检测单网口并确认
- 自动检测路由器 IP (当前默认网关)
- 设置 N100 静态 IP (路由器网段 .1 地址)
- 添加默认路由指向路由器
- 停用 DHCP client (避免 IP 冲突)
- 启用 IP 转发
- 更新 `nftables-single-nic.conf` 和 `docker-compose.yml` 中的接口名

⚠ 执行后 N100 的 IP 会变成静态 (默认 192.168.1.1), 如果已通过 DHCP 连入, 连接会断。去路由器后台确认无误后继续。

Step 4: 加载防火墙 + 启动容器

```
bash 02-nftables-apply.sh
cd /opt/gateway
docker compose up -d
```

Step 5: 配置路由器 DHCP (Cudy WR3600E)

Cudy 保持路由模式 (不切 AP), 修改:

网络 → LAN:

IP: 192.168.1.254
掩码: 255.255.255.0

DHCP 服务器:

DHCP 范围: 192.168.1.100 - 200
默认网关: 192.168.1.1 (N100 IP)
DNS 服务器: 192.168.1.1 (N100 + AdGuard Home)

工作原理:

- 1. 设备 DHCP 拿到网关 = 192.168.1.1 (N100)
- 2. 设备 DNS = 192.168.1.1 (N100 + AdGuard)
- 3. 设备访问外网 → 先经过 N100 (filter 生效) → 再经路由器出去

改动后重启路由器, 所有设备重新获取 DHCP 即生效。

配置 AdGuard Home

同双网口方案 [Step 6](#), 区别:

- 管理界面地址变为 `http://192.168.1.1:3000`
- 不需要开启 AdGuard 内置 DHCP (由路由器担任)

风险与缓解

风险	影响	缓解
N100 宕机	所有设备的网关和 DNS 都指向 N100 → 全网断网	路由器 DHCP 设 备用 DNS 为 223.5.5.5; 切网关需手动去路由器改回 0.0.0.0 (自动探测 N100 存活可写 cron 脚本)
N100 IP 被路由器 DHCP 分配给别人	IP 冲突, 网关不可达	将 N100 的 MAC 在路由器 DHCP 中设为保留/排除, 或 N100 IP 放在 DHCP 范围外
AdGuard 容器未启动	DNS 无法解析	路由器 DHCP 设备用 DNS, 但会绕过过滤 — 建议 <code>systemctl enable docker + restart: unless-stopped</code>

验证

```
bash scripts/single-nic/03-verify.sh
```

单网口模式 vs 双网口模式切换

如需从单网口升级到双网口:

- 1. 停止 nftables: `nft flush ruleset && systemctl stop nftables-gateway`
- 2. 换用 `nftables.conf` (双网口版)
- 3. 运行 `scripts/01-setup-host.sh` (双网口版, 配置 eth0/eth1)
- 4. Cudy 切 AP 模式 + 物理换线
- 5. `scripts/02-nftables-apply.sh` (双网口版)

反向同理: 双网口降级到单网口即可撤下物理改线。

回滚

```
# 停止防火墙
nft flush ruleset
```

```
systemctl stop nftables-gateway
```

```
# 停止容器
```

```
docker compose down
```

```
# 恢复 Cudy 路由模式
```

```
# 登录 Cudy → 开启 DHCP → WAN 口接回光猫 → LAN IP 改回原值
```

```
# 恢复 N100 网口
```

```
# 删除 /etc/netplan/01-gateway.yaml 或恢复备份
```